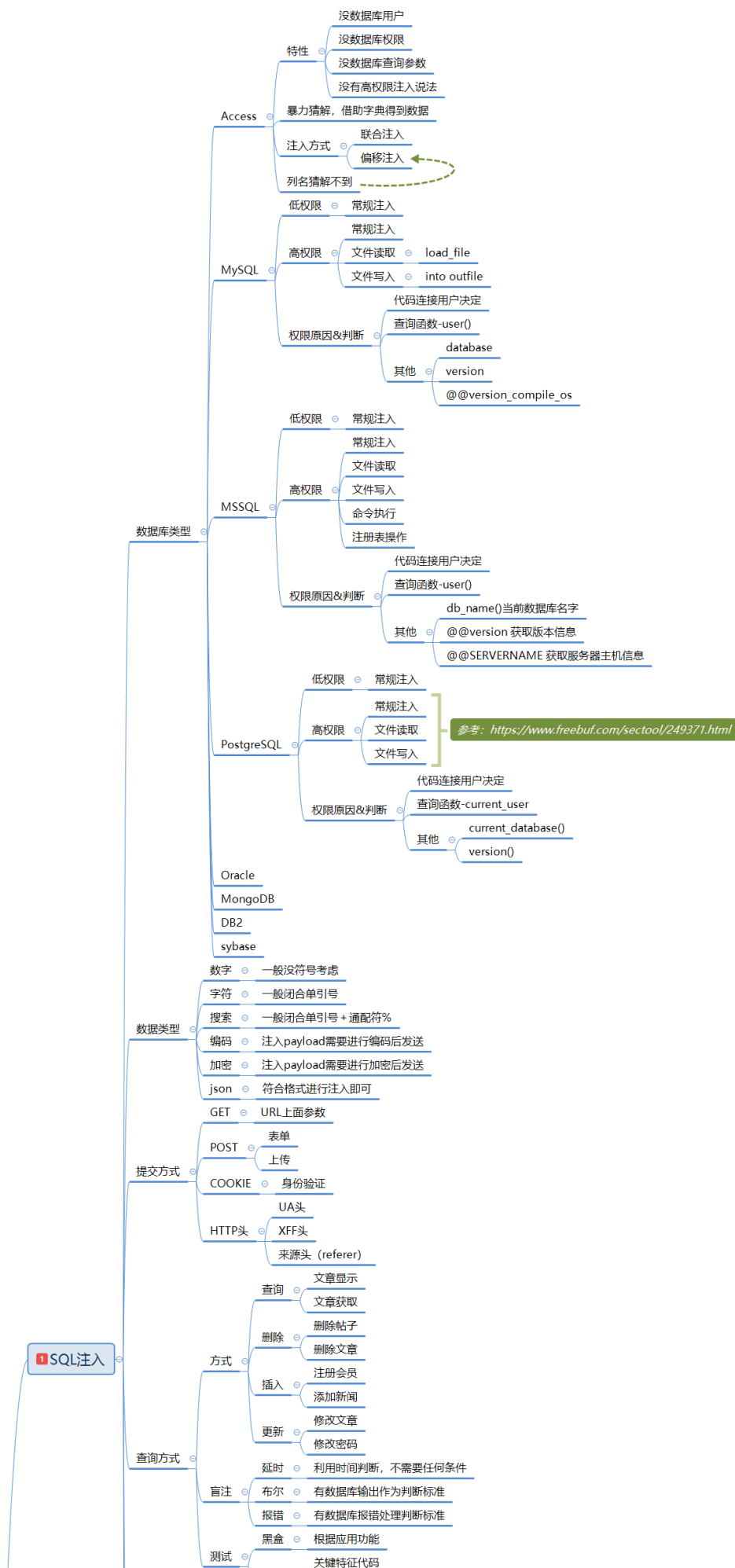
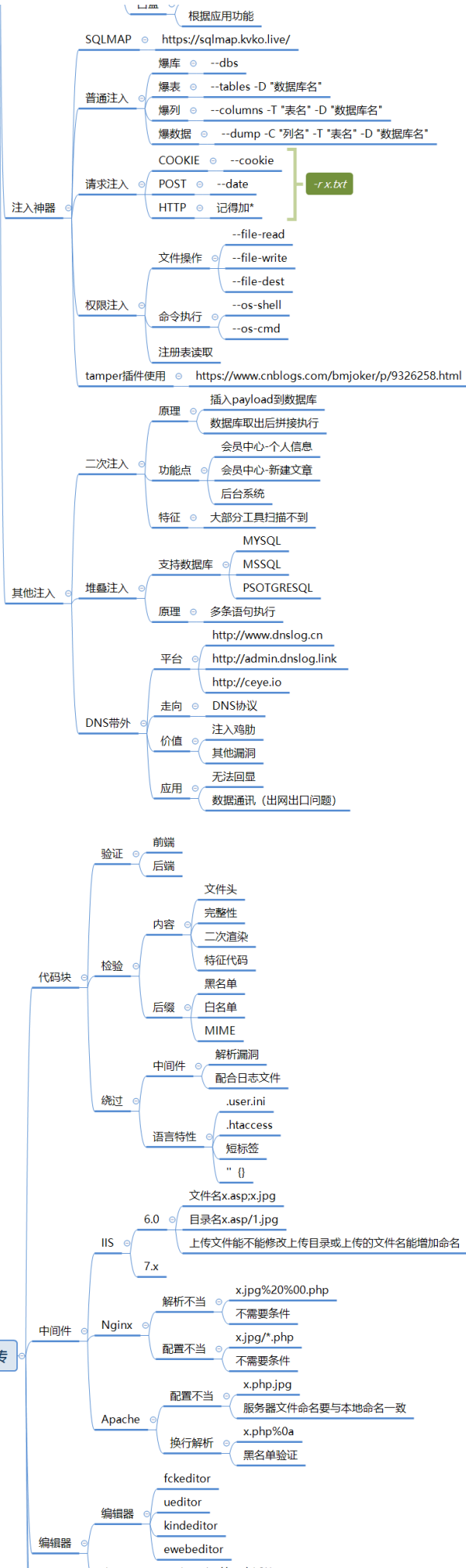
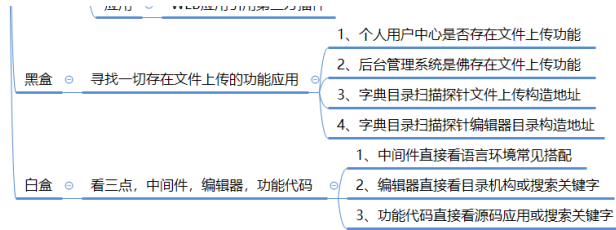


Day56 WEB攻防-CSRF请求伪造&Referer同源&置空&配合XSS&Token值校验&复用删除



2 文件上传





通用安全漏洞

4 CSRF

-
- ```

graph LR
 A[条件] --> B[1、目标用户已经登录了网站，能够执行网站的功能。]
 A --> C[2、目标用户访问了攻击者构造的 URL。]
 C --> D[探针且防御]
 D --> E[1、看验证来源-修复]
 D --> F[2、看凭证有无 token-修复]
 D --> G[3、看关键操作有无验证-修复]
 G --> H[流程]
 H --> I[1、获取目标的触发数据包]
 H --> J[2、利用 CSRFTester 构造导出]
 J --> K[审计]
 K --> L[1、获取目标的触发数据包]
 K --> M[2、利用 CSRFTester 构造导出]

```
- 条件
- 1、目标用户已经登录了网站，能够执行网站的功能。
  - 2、目标用户访问了攻击者构造的 URL。
- 探针且防御
- 1、看验证来源-修复
  - 2、看凭证有无 token-修复
  - 3、看关键操作有无验证-修复
- 流程
- 1、获取目标的触发数据包
  - 2、利用 CSRFTester 构造导出
- 审计
- 1、获取目标的触发数据包
  - 2、利用 CSRFTester 构造导出

## 5 SSRF

- 条件
  - 当前功能应用点利用服务器去解析拟提交的资源
- 功能
  - 分享
  - 转码
  - 翻译
  - 图片加载
  - 收藏
  - 信息探针
  - 内网扫描

| 语言     | PHP                          | Java       | curl                     | Perl              | ASP.NET             |
|--------|------------------------------|------------|--------------------------|-------------------|---------------------|
| http   | ✓                            | ✓          | ✓                        | ✓                 | ✓                   |
| socket | with<br>cURL extensions      | jdk1.5.1_2 | before 7.40.0 不<br>支持SSL | ✓                 | before<br>version 3 |
| ftp    | cURL extensions              | ✓          | before 7.40.0 不<br>支持SSL | ✓                 | ✓                   |
| dns    | ✓                            | ✓          | ✓                        | ✓                 | ✓                   |
| file   | ✓                            | ✓          | ✓                        | ✓                 | ✓                   |
| imap   | ✓                            | ✓          | ✓                        | ✓                 | ✓                   |
| mysql  | with<br>cURL extensions      | ✓          | ✓                        | ✓                 | ✓                   |
| xml    | with<br>cURL extensions      | ✓          | ✓                        | ✓                 | ✓                   |
| xmlrpc | with<br>cURL extensions      | ✓          | ✓                        | ✓                 | ✓                   |
| ldap   | with<br>cURL extensions      | ✓          | ✓                        | ✓                 | ✓                   |
| url2   | 支持于<br>PHP4 and .Japan       | ✓          | ✓                        | 支持于<br>perl 5.005 | ✓                   |
| url3   | 支持于<br>PHP4 and .Japan       | ✓          | ✓                        | ✓                 | ✓                   |
| ldap3  | 支持于<br>perl 5.005 and .Japan | ✓          | ✓                        | ✓                 | ✓                   |
| php    | 支持于<br>PHP4 and .Japan       | ✓          | ✓                        | ✓                 | ✓                   |

- 探针且防御 ○ SSRF常见安全修复防御方案. ○
- 1、禁用跳转
  - 2、禁用不需要的协议
  - 3、固定或限制资源地址
  - 4、错误信息统一-信息处理

- ```

graph LR
    A[审计] --- B[SSRF白盒可能出现的地方]
    A --- C[SSRF黑盒可能出现的地方]
    B --- B1[1. 功能点抓包指向代码块审计]
    B --- B2[2. 功能点函数定位代码块审计]
    C --- C1[1. 社交分享功能]
    C --- C2[2. 转码服务]
    C --- C3[3. 在线翻译]
    C --- C4[4. 图片加载/下载]
    C --- C5[5. 图片/文章收藏功能]
    C --- C6[6. 云服务厂商]
    C --- C7[7. 网站采集, 网站抓取的地方]
    C --- C8[8. 数据库内置功能]
    C --- C9[9. 邮件系统]
    C --- C10[10. 编码处理, 属性信息处理, 文件处理]
    C --- C11[11. 未公开的api实现以及其他扩展调用URL的功能]
    C --- C12[12. 从远程服务器请求资源]
  
```

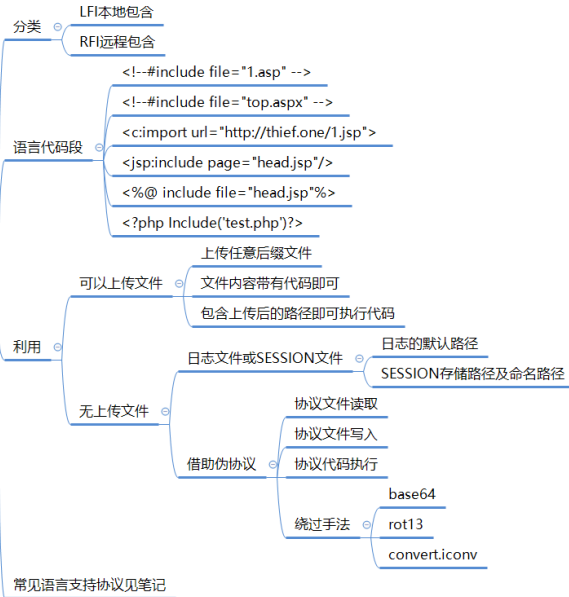
6 XXE&XML

- ```

graph LR
 Root[前置] --- A[理由：传输和存储数据]
 Root --- B[危害：文件读取，端口探针等]
 Root --- C[Content-Type]
 Root --- D[黑盒]
 Root --- E[白盒]
 Root --- F[有回显]
 Root --- G[无回显]
 Root --- H[伪协议玩法]
 Root --- I[禁用dtd实体引用]
 Root --- J[过滤关键词：<!DOCTYPE和<!ENTITY，或者SYSTEM和PUBLIC]
 D --- D1[数据格式]
 D --- D2[svg&excel引用]
 E --- E1[1、可通过应用功能追踪代码定位审计]
 E --- E2[2、可通过脚本特定函数搜索定位审计]
 E --- E3[3、可通过伪协议玩法绕过相关修复等]
 F --- F1[file读取文件]
 F --- F2[http带外访问]
 G --- G1[file读取文件-先读取]
 G --- G2[http带外访问-加载实体]
 G --- G3[dtd实体带外访问-将数据参数发送接收文件接收数据处理]
 H --- H1[见笔记图，后期文件包含会讲]

```

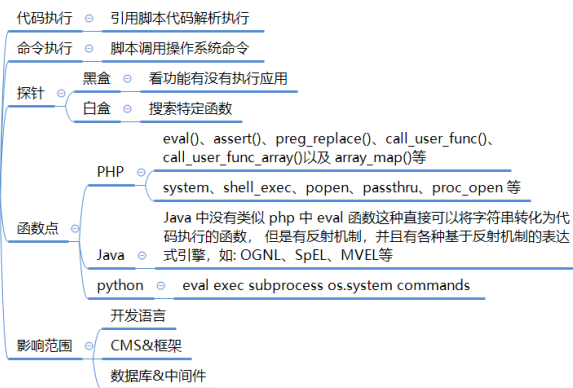
## 文件包含



## 其他文件操作安全



## RCE



## PHP



## 反序列化

### JAVA

#### 属性

- `__toString()`: 把类当做字符串使用时触发
- `__sleep()`: `serialize()`函数会检查类中是否存在一个魔术方法 `__sleep()` 如果存在, 该方法会被优先调用
- `public` (公共的): 在本类内部、外部类、子类都可以访问
- `protect` (受保护的): 只有本类或子类或父类中可以访问
- `private` (私人的): 只有本类内部可以使用
  - 序列化数据显示:
    - `private` 属性序列化的时候格式是 `%00 类名%00 成员名`
    - `protect` 属性序列化的时候格式是 `%00*%00 成员名`

#### 利用

- 单纯的魔术方法逻辑调用
- PHP语言的特性漏洞-如 CVE-2016-7124
- 魔术方法内置原生类
  - 1、获取魔术方法对应原生类
  - 2、利用原生类官方说明配合
- 字符串逃逸&Phar
  - 后面专题讲到

#### 原理

- 序列化: 把 Java 对象转换为字节序列的过程。
- 反序列化: 把字节序列恢复为 Java 对象的过程。

#### 发现

- Serializable Externalizable 接口、fastjson、jackson、gson、ObjectInputStream.read、ObjectObjectInputStream.readUnshared、XMLDecoder.read、ObjectYaml.loadXStream.fromXML、ObjectMapper.readValue、JSON.parseObject 等
- 功能
  - http参数, cookie, session, 存储方式可能是base64(r00), 压缩后的base64(H4s), MII等。Servlets http, Sockets, Session管理器, 包含的协议就包括: JMX, RMI, JMS, JNDI等(\xac \xed)\xmlstream.XmlEncoder等(http Body: Content-type: application/xml) json(jacksonfastjson) http请求中包含
- 数据特性
  - 一段数据以 `r00AB` 开头, 你基本可以确定这串就是 JAVA 序列化 base64 加密的数据。
  - 或者如果以 `aced` 开头, 那么他就是这一段 java 序列化的 16 进制。

#### 利用

- Ysoserial
  - 无组件
  - 有组件
- 特定的组件利用
  - shiro
  - jboos

### Python

#### 解释

- 文件&字符串&字节流&对象相互转换的过程

#### 原理

- 序列化调用魔术方法
- 反序列化调用魔术方法

#### 函数&魔术方法

- 函数使用:
  - `pickle.dump(obj, file)`: 将对象序列化后保存到文件
  - `pickle.load(file)`: 读取文件, 将文件中的序列化内容反序列化为对象
  - `pickle.dumps(obj)`: 将对象序列化成字符串格式的字节流
  - `pickle.loads(bytes_obj)`: 将字符串格式的字节流反序列化为对象
- 魔术方法:
  - `__reduce__()` 反序列化时调用
  - `__reduce_ex__()` 反序列化时调用
  - `__setstate__()` 反序列化时调用
  - `__getstate__()` 序列化时调用

#### 利用

- 注意当前构造版本和目标版本对应
- 注意当前是否需要编码、转换等

#### 白盒

- 搜索特定函数
- 使用bandit

### 访问权限

#### 越权

- 水平同级用户
  - 用户信息获取时未对用户与 ID 比较判断直接查询等
- 垂直低高用户
  - 数据库中用户类型编号接受篡改或高权限操作未验证等

#### 访问控制

- 验证丢失
  - 未包含引用验证代码文件等
- 没有验证
  - 支持空口令、匿名、白名单等

#### 脆弱验证

- Cookie
  - JWT
  - Session
- 不安全的验证逻辑等

### 购买支付

#### 安全点

- 价格和数量
- 产品和订单编号
- 支付模式&接口数据
- 折扣优惠券积分等重复使用&再获取

#### 安全问题

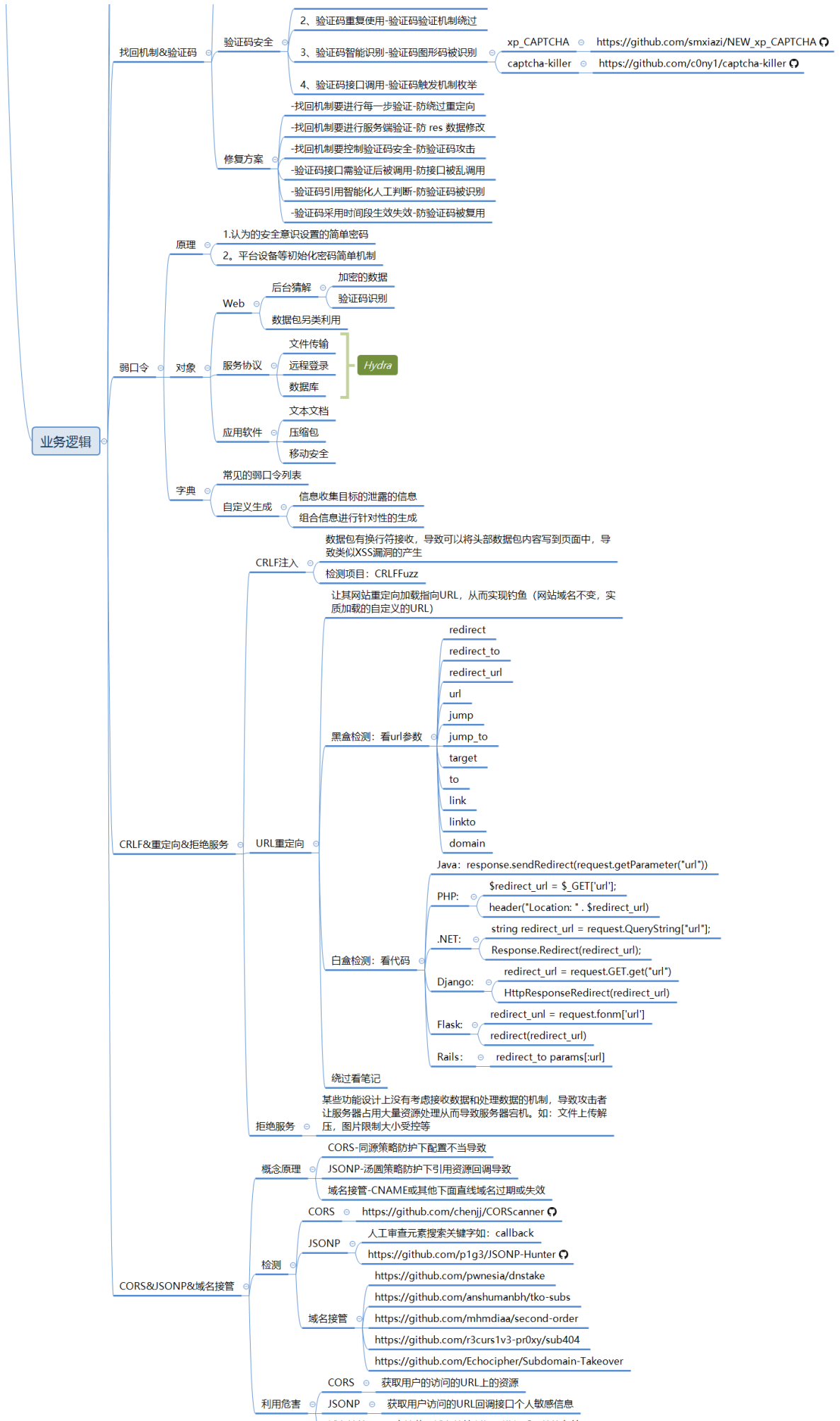
- 1、商品购买-数量&价格&编号等
- 2、支付模式-状态&接口&负数等
- 3、折扣处理-优惠券&积分&重放等

#### 挖掘&修复

- 1、金额以数据库定义为准
  - 2、购买数量限制为正整数
  - 3、优惠券固定使用后删除
  - 4、订单生成后检测对应值
- 功能点抓包造代码测试

#### 找回机制

- 1、用回显状态判断-res 前端判断不安全
- 2、用用户名重定向-修改标示绕过验证
- 3、验证码回显显示-验证码泄漏验证虚设
- 4、验证码简单机制-验证码过于简单爆破





## 1.知识点

- 1、ASP-SQL注入-Access数据库
- 2、ASP-默认安装-数据库泄漏下载
- 3、ASP-IIS-CVE&短文件&解析&写入

- 
- 1、PHP-MYSQL-SQL注入-常规查询
  - 2、PHP-MYSQL-SQL注入-跨库查询
  - 3、PHP-MYSQL-SQL注入-文件读写

- 
- 1、PHP-MYSQL-SQL注入-数据请求类型
  - 2、PHP-MYSQL-SQL注入-数据请求方法
  - 3、PHP-MYSQL-SQL注入-数据请求格式

- 
- 1、PHP-MYSQL-SQL注入-方式增删改查
  - 2、PHP-MYSQL-SQL注入-布尔&延迟&报错
  - 3、PHP-MYSQL-SQL注入-数据回显&报错处理

- 
- 1、PHP-MYSQL-SQL注入-二次注入&利用条件
  - 2、PHP-MYSQL-SQL注入-堆叠注入&利用条件
  - 3、PHP-MYSQL-SQL注入-带外注入&利用条件

- 
- 1、注入工具-SQLMAP-常规猜解&字典配置
  - 2、注入工具-SQLMAP-权限操作&文件命令
  - 3、注入工具-SQLMAP-Tamper&使用&开发
  - 4、注入工具-SQLMAP-调试指纹&风险等级

- 
- 1、PHP-原生态-文件上传-检测后缀&黑白名单
  - 2、PHP-原生态-文件上传-检测信息&类型内容
  - 3、PHP-原生态-文件上传-函数缺陷&逻辑缺陷
  - 4、PHP-原生态-文件上传-版本缺陷&配置缺陷

- 
- 1、PHP-中间件-文件上传-CVE&配置解析
  - 2、PHP-编辑器-文件上传-第三方引用安全

- 3、PHP-CMS源码-文件上传-已知识别到利用

-----

- 1、文件上传-安全解析方案-目录权限&解码还原
- 2、文件上传-安全存储方案-分站存储&OSS对象

-----

- 1、文件包含-原理&分类&危害-LFI&RFI
- 2、文件包含-利用-黑白盒&无文件&伪协议

-----

- 1、文件安全-前后台功能点-下载&读取&删除
- 2、目录安全-前后台功能点-目录遍历&目录穿越

-----

- 1、XSS跨站-输入输出-原理&分类&闭合
- 2、XSS跨站-分类测试-反射&存储&DOM

-----

- 1、XSS跨站-攻击利用-凭据盗取
- 2、XSS跨站-攻击利用-数据提交
- 3、XSS跨站-攻击利用-网络钓鱼
- 4、XSS跨站-攻击利用-溯源综合

-----

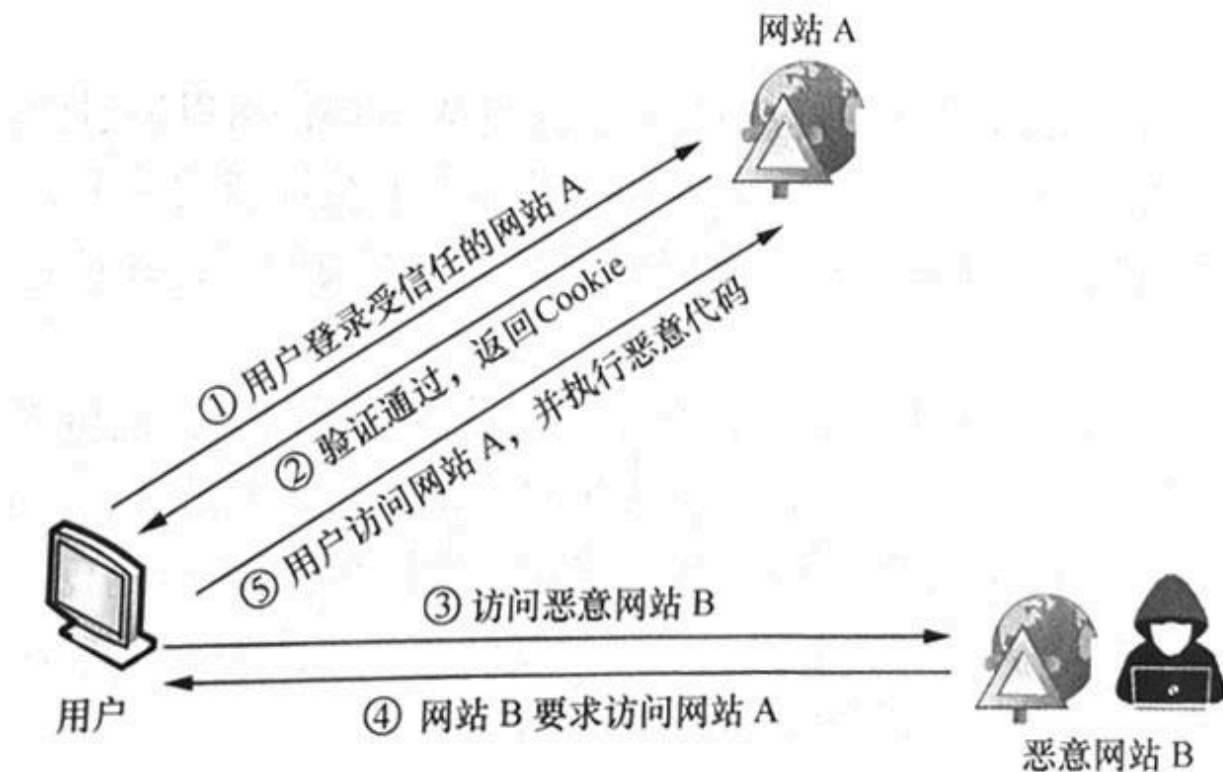
1、XSS跨站-安全防护-CSP策略

2、XSS跨站-安全防护-HttpOnly

3、XSS跨站-安全防护-XSSFilter

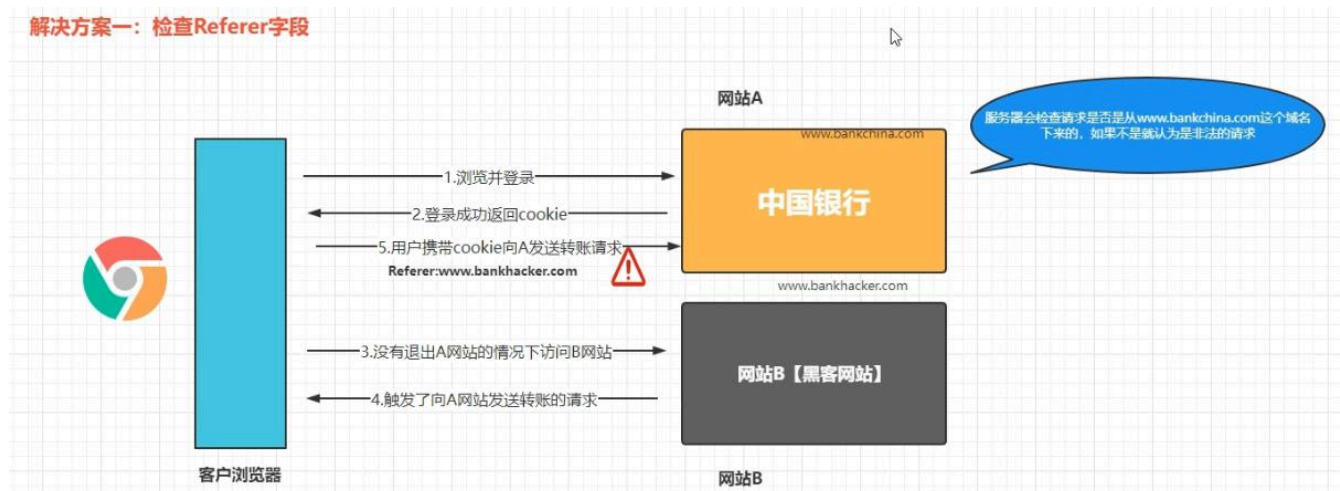
-----

1、CSRF-原理&检测&利用&防御



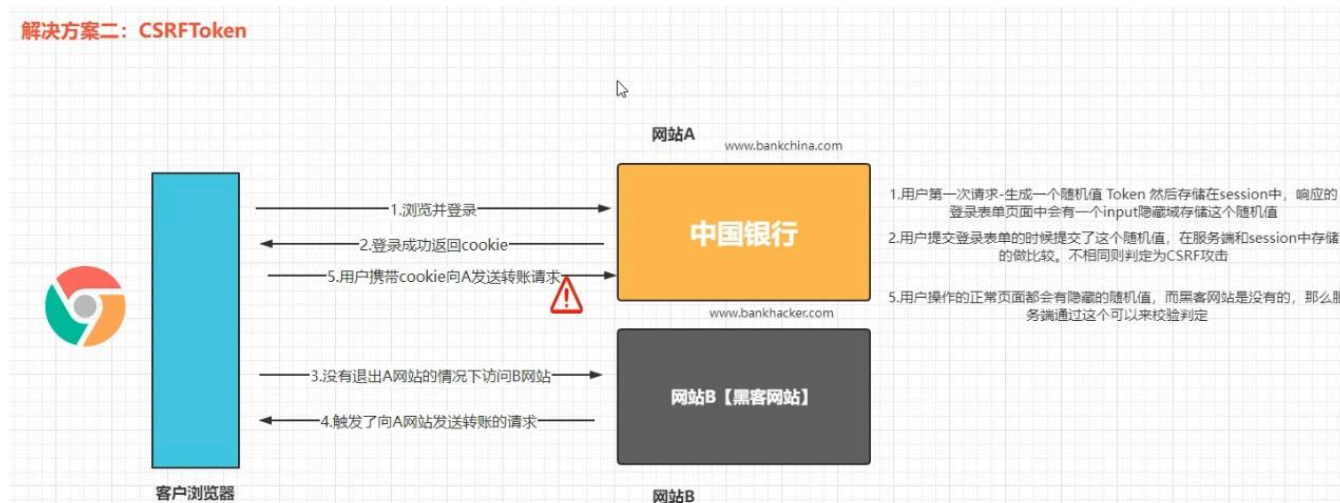
## 2、CSRF-防御-Referer策略隐患

### 解决方案一：检查Referer字段



## 3、CSRF-防御-Token校验策略隐患

### 解决方案二：CSRFToken



## 2.演示案例

### 2.1 CSRF-无检测防护-检测&生成&利用



- 1 检测：黑盒手工利用测试，白盒看代码检验（有无token，来源检验等）
- 2 生成：BurpSuite->Engagement tools->Generate CSRF Poc
- 3 利用：将文件防止自己的站点下，诱使受害者访问（或配合XSS触发访问）

### 2.2 CSRF-Referer同源-规则&上传&XSS



```
1 https://blog.csdn.net/weixin_50464560/article/details/120581841
2 严谨代码PHP DEMO:
3 <?php
4 // 检测来源
5 function checkReferrer() {
6 $expectedReferrer = "http://example.com"; // 期望的来源页面
7
8 if (!isset($_SERVER['HTTP_REFERER']) || $_SERVER['HTTP_REFERER'] !==
9 $expectedReferrer) {
10 die("非法访问");
11 }
12 }
13 // 处理表单提交
14 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
15 // 检测来源
16 checkReferrer();
17
18 // 获取用户输入的数据
19 $name = $_POST['name'];
20 $email = $_POST['email'];
21
22 // 输出用户输入的数据
23 echo "姓名: $name
";
24 echo "邮箱: $email
";
25 exit;
26 }
27 ?>
28
29 <!DOCTYPE html>
30 <html>
31 <head>
32 <title>检测来源示例</title>
33 </head>
34 <body>
35 <h1>检测来源示例</h1>
```

```

36 <form action="<?php echo $_SERVER['PHP_SELF']; ?>" method="POST">
37 <label for="name">姓名: </label>
38 <input type="text" name="name" id="name" required />
39

40 <label for="email">邮箱: </label>
41 <input type="email" name="email" id="email" required />
42

43 <input type="submit" value="提交" />
44 </form>
45 </body>
46 </html>

```

```

47
48 绕过0: 规则匹配绕过问题（代码逻辑不严谨）
49 1、<meta name="referrer" content="no-referrer">
50 2、http://xx.xx.xx.xx/http://xx.xx.xx.xx
51 绕过1: 配合文件上传绕过（严谨使用同源绕过）
52 绕过2: 配合存储XSS绕过（严谨使用同源绕过）

```

## 2.3 CSRF-Token校验-值删除&复用&留空



```

1 https://blog.csdn.net/weixin_50464560/article/details/120581841
2 严谨代码PHP DEMO:
3 <?php
4 session_start();
5
6 // 生成并存储 CSRF Token
7 function generateCSRFToken() {
8 $token = bin2hex(random_bytes(32));
9 $_SESSION['csrf_token'] = $token;
10 return $token;
11 }
12
13 // 检查 CSRF Token 是否有效
14 function validateCSRFToken($token) {
15 return isset($_SESSION['csrf_token']) && $_SESSION['csrf_token'] === $token;
16 }
17
18 // 处理表单提交
19 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
20 // 检查 CSRF Token
21 if (!isset($_POST['csrf_token']) || !validateCSRFToken($_POST['csrf_token'])) {
22 die("CSRF Token 验证失败");
23 }
24
25 // 获取用户输入的数据
26 $name = $_POST['name'];
27 $email = $_POST['email'];

```

```

28
29 // 输出用户输入的数据
30 echo "姓名: $name
";
31 echo "邮箱: $email
";
32 exit;
33 }
34
35 // 生成 CSRF Token
36 $csrfToken = generateCSRFToken();
37 ?>
38
39 <!DOCTYPE html>
40 <html>
41 <head>
42 <title>CSRF Token 示例</title>
43 </head>
44 <body>
45 <h1>CSRF Token 示例</h1>
46 <form action="<?php echo $_SERVER['PHP_SELF']; ?>" method="POST">
47 <input type="hidden" name="csrf_token" value="<?php echo $csrfToken; ?>" />
48 <label for="name">姓名: </label>
49 <input type="text" name="name" id="name" required />
50

51 <label for="email">邮箱: </label>
52 <input type="email" name="email" id="email" required />
53

54 <input type="submit" value="提交" />
55 </form>
56 </body>
57 </html>
58
59 绕过0: 将Token参数值复用（代码逻辑不严谨）
60 绕过1: 将Token参数删除（代码逻辑不严谨）
61 绕过2: 将Token参数值置空（代码逻辑不严谨）

```

## 资源:

- 1 **CSRF的绕过方法与利用:**
- 2 [https://blog.csdn.net/weixin\\_50464560/article/details/120581841](https://blog.csdn.net/weixin_50464560/article/details/120581841)